Relazione Parallel Computing

Traccia: Boids

Aharram Youness 7198778

Abstract

L'algoritmo *Boids* modella il movimento collettivo di agenti autonomi, come gli stormi di uccelli, basandosi su regole locali di separazione, allineamento e coesione. Nella sua implementazione naïve, la simulazione su larga scala presenta una complessità computazionale quadratica, rappresentando un severo collo di bottiglia. In questo progetto è stata sviluppata una versione fortemente ottimizzata sia sequenziale che parallela. Lo studio si concentra sull'analisi di diverse strategie di parallelizzazione, valutando l'impatto del numero di thread, delle politiche di scheduling, della località dei dati e della vettorizzazione. I risultati mostrano un drastico incremento delle prestazioni rispetto alla baseline sequenziale, evidenziando un'ottima scalabilità. L'approccio dimostra come la combinazione di multithreading e ottimizzazione della memoria possa accelerare efficacemente la simulazione. Nella configurazione ottimale, il sistema ha registrato uno speedup massimo pari a 7.98×.

Introduzione

L'algoritmo *Boids* prevede che ogni agente (boid) aggiorna iterativamente la propria posizione e velocità in base a tre regole cinematiche locali: **Separazione** (allontanarsi per evitare collisioni entro una distanza critica), **Allineamento** (sincronizzare il proprio vettore velocità con la media del vicinato) e **Coesione** (convergere verso il centro di massa locale). Nella sua forma più semplice, l'algoritmo impone che ogni boid valuti la propria distanza da tutti gli altri, portando a una complessità temporale quadratica che rende impraticabile la simulazione in tempo reale all'aumentare del numero di agenti. Per risolvere tale problema, è stata introdotta una struttura di partizionamento spaziale (griglia uniforme) che riduce le interazioni al solo vicinato locale. Tuttavia, emergono due ulteriori e severi colli di bottiglia a livello hardware:

- **Memory Wall e Cache Miss:** L'elevato e imprevedibile numero di accessi sparsi in memoria per recuperare i dati dei boids adiacenti penalizza drasticamente la località spaziale dei dati e l'efficienza della cache.

- **Branch Prediction Penalty:** La presenza di numerosi salti condizionali (istruzioni `if` per calcolare distanze, bordi e campi visivi) interrompe costantemente la pipeline di istruzioni della CPU e ostacola la vettorizzazione del codice.

L'obiettivo principale di questo progetto è affrontare e superare queste sfide computazionali, sviluppando e analizzando una versione parallela fortemente ottimizzata dell'algoritmo *Boids* in ambiente C. Per isolare e misurare correttamente i miglioramenti, il progetto include lo sviluppo di una versione sequenziale ottimizzata con griglia, utilizzata come baseline per il calcolo dello speedup. Il progetto include inoltre un'analisi prestazionale finalizzata a valutare la scalabilità e l'efficienza del sistema, nonché l'impatto di diversi fattori chiave. In particolare, vengono esaminate le politiche di scheduling, il confronto tra differenti organizzazioni dei dati in memoria (Structure of Arrays vs Array of Structures) e il comportamento dell'algoritmo in scenari di *strong scaling* e *weak scaling*. L'analisi considera anche l'influenza della dimensione del *chunk* sulle prestazioni complessive, al fine di individuare le configurazioni più efficienti.

Sequential Implementation

La versione sequenziale dell'algoritmo Boids è stata implementata in due varianti principali:

- **Versione naïve:** ogni boid interagisce con tutti gli altri, con complessità $O(n^2)$.
- **Versione ottimizzata con griglia:** le interazioni sono limitate ai boid presenti nella stessa cella e nelle 8 celle adiacenti, riducendo significativamente la complessità.

Entrambe le varianti condividono la stessa logica di aggiornamento, differendo unicamente nel metodo di ricerca dei vicini.

Struttura dell'algoritmo

L'algoritmo si articola in tre fasi principali: inizializzazione, aggiornamento iterativo e rendering (opzionale).

Inizializzazione

Ogni boid è inizializzato con posizione (x, y) e velocità (v_x, v_y) generate in maniera deterministica utilizzando un generatore lineare congruenziale (LCG). Questo garantisce la riproducibilità dei risultati tra esecuzioni. L'inizializzazione genera valori diversi per ogni boid a partire da un seed comune.

Aggiornamento temporale

Ad ogni passo temporale, per ogni boid i si eseguono le seguenti operazioni:

Versione naïve:

1. Calcolare la distanza al quadrato tra il boid i e tutti gli altri boid $j \neq i$.
2. Applicare la regola di separazione se la distanza è inferiore a un determinato raggio chiamato protected range.
3. Applicare le regole di coesione e allineamento se la distanza è inferiore ad un determinato raggio chiamato visual range, accumulando posizione e velocità medie dei vicini.
4. Aggiornare velocità e posizione del boid, limitando la velocità minima e massima e gestendo eventuali bordi dello spazio di simulazione.

Versione con griglia:

1. Suddividere lo spazio in celle
2. Assegnare ogni boid alla cella corrispondente.
3. Per ogni boid i, considerare solo i boid nella stessa cella e nelle 8 celle adiacenti per calcolare separazione, coesione e allineamento.
4. Aggiornare velocità e posizione come nella versione naïve.

Rendering e doppio buffer

Per garantire la coerenza dei dati durante la simulazione, l'algoritmo utilizza due buffer distinti per memorizzare lo stato dei boid: `flock` e `next_flock`.

- **Lettura:** Tutte le regole di aggiornamento dei boid (separazione, allineamento, coesione) leggono lo stato corrente dai valori presenti in `flock`.
- **Scrittura:** I risultati dei calcoli, ovvero le nuove posizioni e velocità, vengono scritti in `next_flock`.

Al termine di ogni passo temporale, i due buffer vengono scambiati (`flock` $\leftrightarrow$ `next_flock`), in modo che il nuovo stato diventi quello corrente per il passo successivo. Questa strategia consente di:

1. Evitare conflitti tra lettura e scrittura durante l'aggiornamento dei boid.
2. Garantire che ogni boid calcoli le interazioni rispetto allo stato consistente degli altri agenti.
3. Supportare agevolmente il rendering in tempo reale, poiché il buffer corrente (`flock`) contiene sempre lo stato coerente dei boid.

Identificazione delle sezioni parallelizzabili

L'algoritmo Boids presenta diverse opportunità di parallelizzazione grazie alla natura indipendente dei calcoli per ciascun boid. Le principali sezioni parallelizzabili sono:

1. **Costruzione della griglia spaziale:** La fase di mappatura spaziale e inserimento dei boid nelle celle può essere parallelizzata distribuendo i boid tra i thread. Tuttavia, affinché l'incremento del contatore di una specifica cella sia *thread-safe*, è necessario proteggere le scritture tramite operazioni hardware atomiche, evitando così *race conditions*.

2. **Ricerca dei vicini e aggiornamento:** Queste due fasi avvengono simultaneamente. Una volta costruita la griglia (che diventa a sola lettura), il calcolo dell'interazione di ciascun boid con il vicinato e il conseguente aggiornamento della sua posizione e velocità sono operazioni totalmente indipendenti per ogni boid.

La fattibilità di questa parallelizzazione lock-free nel *core* computazionale è garantita dall'utilizzo di un **doppio buffer** (`flock` e `next_flock`). Poiché le letture dello stato avvengono da un buffer e le scritture del nuovo stato su un buffer separato, non sussistono dipendenze di dati tra le iterazioni. In sintesi, tutte le operazioni di aggiornamento rappresentano sezioni ideali per il multi-threading (tramite `#pragma omp parallel for`) e per la **vettorizzazione SIMD**, mentre la scrittura nella griglia è l'unica fase sensibile al contesto concorrente.

Implementazione Parallela

La transizione dalla versione sequenziale alla versione parallela ha richiesto una riprogettazione dell'architettura del software. L'obiettivo primario è stato massimizzare il *throughput* computazionale sfruttando il multi-threading tramite OpenMP, minimizzando al contempo l'overhead di sincronizzazione.

Data Structures e Memory Layout: AoS vs SoA

La prima sfida affrontata nella parallelizzazione ha riguardato l'organizzazione dei dati in memoria. L'approccio **AoS (Array of Structures)**, in cui ogni boid è una 'struct' contenente posizione e velocità, sebbene intuitivo, si è rivelato estremamente inefficiente. Il caricamento di una singola coordinata x in un registro della CPU porta con sé in cache anche dati non necessari in quel momento (come le velocità), sprecando banda di memoria (*memory bound*) e ostacolando la vettorizzazione. Per risolvere questo problema, l'architettura è stata riscritta adottando la struttura **SoA (Structure of Arrays)**.

```
typedef struct {
    float* x; float* y; float* vx; float* vy;
    int n;
} Boids;
```

Nel layout SoA, tutte le coordinate x di tutti i boid sono contigue in memoria, così come le y, le v_x e le v_y. L'allocazione di questi array è stata effettuata forzando l'allineamento a 64 byte, garantendo che i dati si incastrino perfettamente nelle *cache lines* della CPU. Questo approccio garantisce letture perfettamente lineari e prepara il terreno per l'elaborazione vettoriale.

Strategia di Parallelizzazione e Direttive OpenMP

La parallelizzazione è stata implementata utilizzando lo standard **OpenMP** Le principali direttive utilizzate sono:

- `#pragma omp parallel for schedule(runtime)`: Applicata ai cicli principali di inizializzazione e aggiornamento (`update_boids` e `update_boids_grid`). La clausola `schedule(runtime)` è stata fondamentale per impostare a runtime la scelta della politica di bilanciamento del carico (*static, dynamic, guided*).

- `#pragma omp parallel for collapse(2)`: Utilizzata per l'azzeramento e l'inizializzazione della griglia spaziale. Questa clausola ordina al compilatore di fondere i due cicli annidati (es. iterazione su righe e colonne) in un unico spazio di iterazione lineare. Ciò incrementa drasticamente il grado di parallelismo esponibile, garantendo che anche con griglie di dimensioni ridotte ci sia un numero sufficiente di *task* indipendenti per saturare tutti i thread disponibili.

Meccanismi di Sincronizzazione

L'elaborazione concorrente ha richiesto l'implementazione di rigidi meccanismi di sincronizzazione per evitare *race conditions*:

1. **Aggiornamento dello stato (Double Buffering):** Come nella versione sequenziale, i thread leggono dallo stato condiviso `flock` e scrivono concorrentemente nel buffer indipendente `next_flock`. Nessuna sincronizzazione (es. `critical` o `barrier`) è necessaria in questa fase, eliminando di fatto i colli di bottiglia architetturali.

2. **Costruzione della griglia:** Durante il popolamento della griglia, più thread potrebbero tentare di inserire contemporaneamente boid diversi nella medesima cella. Questo conflitto è stato risolto utilizzando l'istruzione `atomic_fetch_add` sul contatore della cella (`cell->count`), garantendo inserimenti hardware *thread-safe* con un overhead minimo.

3. **Pipeline e GUI:** Per la visualizzazione real-time, un *worker thread* dedicato fa avanzare la simulazione, mentre il *main thread* si occupa del rendering. Lo scambio dei puntatori a fine frame è protetto da un `pthread_mutex_t` per garantire la coerenza del rendering.

Sfide incontrate e soluzioni

l'utilizzo della struttura SoA non si sono rivelati sufficienti per avere prestazioni superiori alla versione AoS, a causa di due ostacoli.

Branch Prediction Penalty e Vettorizzazione

Problema: L'algoritmo Boids originale fa largo uso di istruzioni condizionali (es. `if (dist < VISUAL_RANGE)`) per decidere quali vicini includere nel calcolo. Nelle architetture multi-core vettoriali, questi salti condizionali causano continui svuotamenti della pipeline e impediscono al compilatore di emettere istruzioni SIMD per processare più boid in una singola operazione di clock. **Soluzione (Branchless Programming):** Il codice è stato riscritto eliminando i rami condizionali critici nel loop interno. Le condizioni `if` sono state trasformate in "maschere matematiche" booleane (0.0 o 1.0). Abbinata al flag di compilazione `-ffast-math`, questa logica ha costretto la CPU a elaborare i calcoli linearmente, abilitando una perfetta vettorizzazione automatica SIMD e moltiplicando il *throughput* delle operazioni in virgola mobile.

Decadimento della Località Spaziale

Problema: Con l'introduzione della griglia spaziale, il layout SoA incontra un limite fisiologico. Sebbene i dati di *tutti* i boid siano contigui, i boid che risiedono nella *stessa cella fisica* non lo sono necessariamente in memoria (es. boid 1 e boid 900 potrebbero essere vicini visivamente, ma distanti in RAM). Questo causa continui salti nell'accesso agli array, penalizzando enormemente le prestazioni. **Soluzione (Lazy Spatial Sorting):** Per garantire che l'adiacenza nello spazio simulato corrisponda a un'adiacenza fisica in memoria, è stato introdotto un algoritmo di *Counting Sort* lineare basato sull'indice della cella. Tuttavia, ordinare la memoria ad ogni iterazione comporterebbe un costo eccessivo. È stata quindi implementata una strategia di **ordinamento ammortizzato**: l'ordinamento spaziale viene eseguito solo periodicamente (es. ogni 15 frame) utilizzando un buffer temporaneo (`scratch_flock`) e uno *swap a tre puntatori* per non interrompere il ciclo del doppio buffer. Questa tecnica ibrida permette all'architettura SoA di esprimere tutto il suo potenziale multi-core e superare agilmente i limiti dell'approccio AoS.

Setup

Specifiche hardware

Gli esperimenti sono stati eseguiti su una macchina con la seguente configurazione:

- **CPU**: Intel Core i7-1280P (12th Generation)
- **Core**: 14 core fisici, 20 thread (Hyper-Threading)

- **Frequenza**: fino a 4.8 GHz
- **Cache**:
  - L1: 544 KB (data), 704 KB (instruction)
  - L2: 11.5 MB
  - L3: 24 MB
- **Memoria RAM**: 16 GB

Ambiente software

- **Sistema operativo**: Ubuntu 24.04.2 LTS
- **Compilatore**: GCC 13.3.0
- **Librerie**: OpenMP (parallelizzazione), Raylib (visualizzazione)

Il codice è stato compilato utilizzando i seguenti flag:

- `-O3`: Abilitazione del massimo livello di ottimizzazione aggressiva.
- `-march=native`: Autorizzazione al compilatore a generare codice macchina specifico per l'architettura host, sbloccando i set di istruzioni SIMD.
- `-ffast-math`: per abilitare il riordino delle operazioni in virgola mobile e massimizzare la vettorizzazione dei calcoli trigonometrici e geometrici.
- `-fopenmp`: Attivazione delle direttive pragma per il supporto al multi-threading.

Queste opzioni consentono di sfruttare sia il parallelismo multi-core sia la vettorizzazione SIMD automatica.

Parameters Tested

Gli esperimenti sono stati condotti variando:

- **Dimensioni del Problema:** Sono stati simulati stormi di dimensioni eterogenee, partendo da scenari leggeri ($N = 1.000$) fino a carichi di stress per la CPU e la memoria ($N = 10.000, 20.000$).
- **Parametri di Parallelizzazione:** Per valutare la scalabilità e il comportamento dell'implementazione parallela, il numero di thread è stato incrementato progressivamente, esplorando l'insieme $T \in \{1, 2, 4, 8, 14, 16, 20, 32\}$.
- **Politiche di Scheduling OpenMP:** Per avere un analisi completa sono stati testati gli scheduler `static`, `dynamic` e `guided`.
- **Layout di Memoria:** E' stato fatto un confronto diretto tra il layout *AoS* (Array of Structures) e *SoA* (Structure of Arrays).

Metodologia di Misurazione

Per misurare le prestazioni in modo affidabile ed evitare che il sistema operativo falsasse i risultati, i test sono stati eseguiti seguendo questi passaggi:

1. **Riscaldamento (Warm-up):** Prima di far partire il cronometro, la simulazione esegue 5 frame a vuoto. Questo serve a dare il tempo alla CPU di entrare in Turbo Boost e a preparare i thread di OpenMP. Nelle versioni SoA, questo ci permette anche di fare un primo ordinamento spaziale dei dati in memoria.

2. **Misurazione a blocchi:** Misurare il tempo di un singolo frame è rischioso perché i microscatti del computer sballano i numeri. Per questo, ogni test misura il tempo totale necessario per completare un grosso blocco di 1000 iterazioni.

3. **Misurazione dei tempi:** Per misurare il tempo reale impiegato (*Wall-clock time*) abbiamo usato la funzione `omp_get_wtime()`. Alla fine di tutti i run, abbiamo calcolato il tempo medio, il minimo, il massimo e la deviazione standard.

Execution Configuration

La configurazione di esecuzione è stata adattata al contesto:

- In modalità benchmark sono stati utilizzati tutti i thread disponibili;
- In modalità grafica è stato lasciato un thread libero per il rendering.

Validazione della Correttezza

Prima di procedere all'analisi prestazionale, è stato fondamentale garantire che le ottimizzazioni introdotte (SoA, vettorizzazione branchless, multithreading) non alterassero la semantica del modello originale. La validazione è stata effettuata tramite un approccio deterministico:

1. **Inizializzazione controllata:** Sia la versione sequenziale che la versione parallela SoA sono state inizializzate con il medesimo *seed* deterministico.

2. **Confronto dello stato finale:** Dopo un numero prefissato di *frame* di simulazione (es. 1000 iterazioni), lo stato finale dello stormo (posizioni e velocità di tutti i boid) generato dalla versione parallela è stato confrontato con quello della baseline sequenziale.

Confronto prestazioni: sequenziale vs parallela SoA

La prima anaisi effettuta consistee nel confrontare i tempi di esecuzione della versione parallela (basata su layout SoA e logica branchless) con la *baseline* sequenziale ottimizzata tramite griglia spaziale.

Analisi del Wall-Clock Time

La misurazione primaria si basa sul *wall-clock time* (tempo reale di esecuzione) espresso in secondi.

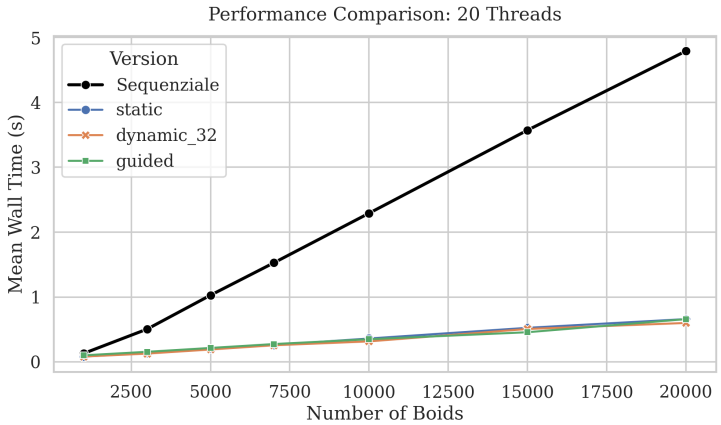

Figure 1: Confronto del Wall-Clock Time. La linea nera rappresenta l'implementazione sequenziale, mentre le linee colorate mostrano l'implementazione parallela SoA al variare dello scheduler.

Come si evince dalla Figura 1, l'implementazione sequenziale presenta una crescita dei tempi molto rapida all'aumentare del carico (N_{boids}). Ad esempio, nello scenario di stress massimo ($N = 20.000$), la CPU impiega oltre 4.7 secondi per completare l'elaborazione. L'attivazione della pipeline parallela abbatte drasticamente questa latenza, mantenendo il sistema in regime interattivo (sotto il secondo) anche per carichi massivi, dimostrando la superiorità dell'approccio parallelo

Calcolo dello Speedup

La Tabella 1 riassume i picchi prestazionali registrati. Nelle configurazioni ad alto carico, l'integrazione di OpenMP, vettorizzazione SIMD e *Lazy Spatial Sorting* ha permesso di raggiungere uno speedup di picco pari a **7.98x** con tecnica di scheduling *dynamic*.

Table 1: Speedup

Boids	2 Threads	4 Threads	8 Threads	16 Threads	20 Threads	32 Threads
1000	1.77 (G)	2.34 (G)	2.37 (D)	1.90 (D)	1.60 (D)	0.83 (S)
3000	2.11 (G)	3.28 (G)	4.12 (D)	4.16 (D)	3.95 (D)	2.47 (D)
5000	2.41 (G)	3.91 (G)	4.95 (D)	5.76 (D)	5.38 (D)	3.76 (D)
7000	2.47 (G)	3.97 (G)	5.32 (D)	6.13 (D)	5.96 (D)	4.59 (D)
10000	2.48 (D)	4.12 (G)	5.76 (D)	6.80 (G)	7.25 (D)	5.48 (D)
15000	2.75 (G)	4.32 (G)	5.96 (D)	7.64 (D)	7.81 (G)	6.10 (D)
20000	2.63 (D)	4.38 (D)	6.22 (D)	7.65 (D)	7.98 (D)	7.08 (D)

Legenda: (S) Static, (D) Dynamic, (G) Guided

Strong e weak scaling

Analisi di Strong Scaling e Limiti Architetturali

Per valutare la capacità del sistema di abbattere i tempi di calcolo distribuendo un carico fisso, è stato condotto un esperimento di **Strong Scaling** simulando uno stormo di 15.000 boids. I test hanno esplorato un range da 2 a 64 thread, confrontando le tre principali politiche di scheduling di OpenMP (`static`, `dynamic_32`, `guided`).

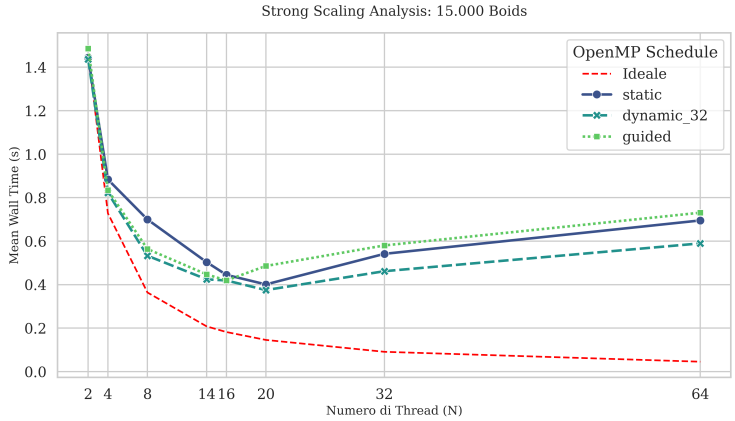

Figure 2: Strong Scaling per $N = 15.000$. Il tempo di esecuzione decresce proporzionalmente all'aumentare dei thread, raggiungendo il minimo globale a 20 thread, per poi subire un degrado a causa dell'oversubscription.

I dati in Figura 2 (relativi al caso con 15,000 boids) mostrano come il tempo di esecuzione *wall-clock* decresca in modo significativo all'aumentare dei thread. Il picco prestazionale assoluto viene registrato esattamente a 20 thread utilizzando la politica `dynamic` (0.37 s). È da notare come la politica di scheduling `dynamic`, a differenza delle altre due, fornisca sempre la performance migliore

per ogni configurazione di thread testata. Un dato di estremo interesse è il comportamento del sistema in regime di *oversubscription* (32 e 64 thread). Creare un numero di *thread* superiore ai core hardware disponibili causa un degrado prestazionale tangibile in tutte le politiche di scheduling, con tempi che tornano a salire. Questo fenomeno è imputabile al costo del *context switching* imposto dal sistema operativo e all'aumento della contesa per le risorse di cache condivise.

Analisi di Weak Scaling e Densità Spaziale

Il **Weak Scaling** valuta l'efficienza dell'algoritmo nel gestire problemi di dimensioni crescenti mantenendo costante il carico di lavoro assegnato al singolo thread. In questo esperimento, utilizzando lo scheduler `dynamic,32`, la dimensione del problema è stata scalata in modo proporzionale ai thread (circa 1.000 boids per thread aggiuntivo), partendo da 2.000 boids su 2 thread fino a 32.000 boids su 32 thread.

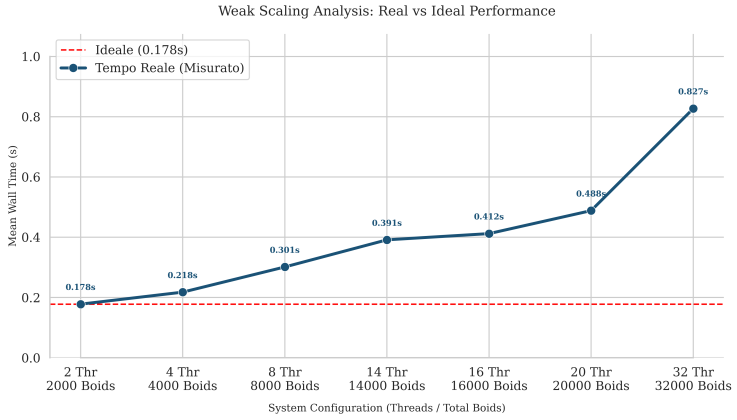

Figure 3: Weak Scaling (1.000 boids per thread). L'aumento fisiologico del tempo è dovuto all'incremento della densità spaziale dello stormo nel dominio di simulazione fisso.

In un sistema teorico a parallelismo perfetto, la curva del *wall-clock time* in *weak scaling* dovrebbe presentarsi come una linea retta orizzontale. Come illustrato in Figura 3, il tempo di esecuzione subisce invece un graduale incremento, passando da 0.18 s (2 thread) a 0.82 s (32 thread). Questo comportamento non è indicativo di un difetto di parallelizzazione, ma deriva dalla natura geometrica del problema. L'algoritmo con griglia spaziale ha una complessità pari a $O(N \cdot M)$, dove M è il numero di vicini locali. Aumentando N all'interno di un dominio spaziale fisso (la finestra di rendering), la densità dello stormo aumenta proporzionalmente. Di conseguenza, pur mantenendo costante il numero di boids totali gestiti da un singolo thread, il numero di interazioni locali M calcolate per ogni iterazione aumenta, giustificando la salita della curva.

Ottimizzazione della Granularità: Analisi del Chunk Size

La scelta della granularità impone un compromesso fondamentale nel calcolo parallelo:

- **Chunk piccoli:** Favoriscono uno bilanciamento del carico quasi perfetto, ma introducono un severo *overhead* di sincronizzazione, poiché i thread devono interrogare continuamente la coda di lavoro globale di OpenMP protetta da lock.
- **Chunk grandi:** Minimizzano l'overhead dello scheduler, ma espongono al rischio di sbilanciamento. Se un E-core (più lento) acquisisce un blocco troppo grande verso la fine dell'iterazione, i P-cores (più veloci) rimarranno in attesa.

Per quantificare questo fenomeno, è stato eseguito un test mirato su uno stormo di 10.000 boid utilizzando 20 thread, facendo variare la dimensione del chunk lungo le potenze di due ($1 \leq \text{chunk} \leq 128$).

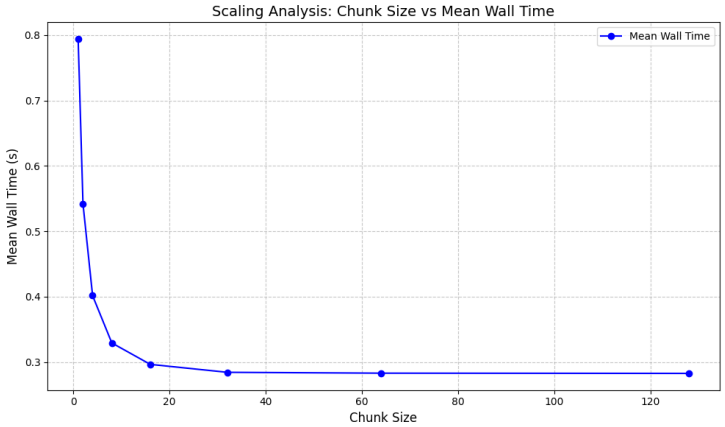

Figure 4: Impatto del Chunk Size sul tempo di esecuzione con scheduler dinamico ($N = 10.000$, $T = 20$).

I risultati sperimentali (Figura 4) mostrano una discesa iniziale molto netta. Se si divide il lavoro assegnando un solo elemento alla volta (`chunk=1`), il sistema fa registrare il tempo peggiore in assoluto (0.794 s): lo sforzo speso per distribuire continuamente pezzettini così piccoli richiede più tempo del lavoro stesso, annullando di fatto i vantaggi dei thread multipli. All'aumentare della dimensione del blocco di lavoro, i tempi crollano rapidamente. Arrivati ad una dimensione del chunk pari a 32, la situazione si stabilizza: da qui in poi i tempi rimangono bassi e costanti fino a `chunk=128`, dove si registra il tempo minimo assoluto (0.283 s). Il grafico mostra che, in questo caso, non si osserva un peggioramento utilizzando blocchi di grandi dimensioni. La divisione del lavoro

è sufficientemente corposa da non sprecare tempo nella sua gestione, ma non così ampia da causare uno sbilanciamento, evitando che alcuni thread terminino troppo presto rimanendo inattivi nella fase finale del calcolo.

Impatto delle Politiche di Scheduling in Regime di Alto Carico

Al fine di isolare e valutare criticamente il comportamento degli algoritmi di *scheduling* messi a disposizione da OpenMP (`static`, `dynamic` e `guided`), l'analisi è stata focalizzata sullo scenario a massimo carico, corrispondente a 20.000 boid.

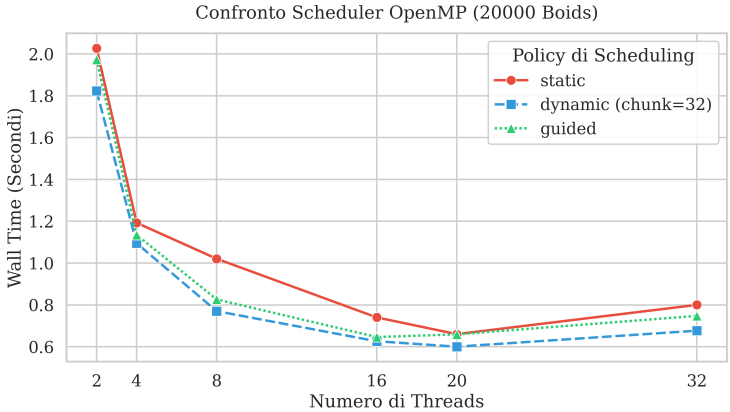

Figure 5: Andamento del Wall-Clock Time per 20.000 boid al variare della politica di scheduling.

Osservando l'andamento delle curve (Figura 5), si nota come il comportamento del sistema si evolva all'aumentare della concorrenza. Inizialmente, con un numero ridotto di thread (tra 2 e 4), le abbondanti risorse computazionali permettono ai tempi di esecuzione di decrescere in modo netto. In questa prima fase, le tre curve procedono vicine, anche se la politica `dynamic` inizia già a mostrare un leggero vantaggio. La dinamica cambia man mano che il sistema satura i core fisici e logici del processore (da 8 a 16 thread): qui i limiti dell'assegnazione statica su un carico di lavoro potenzialmente sbilanciato emergono in modo evidente, con la politica `static` che rimane notevolmente indietro rispetto alle altre due. Al punto di saturazione ottimale (20,000 boid a 20 thread), le prestazioni di `static` e `guided` convergono in un sostanziale pareggio (rispettivamente 0.660 s e 0.659 s). Tuttavia, la politica `dynamic` conferma la sua superiorità distaccando entrambe e segnando il picco prestazionale assoluto (0.600 s). Il reale punto di rottura emerge spingendo il sistema in regime di *oversubscription* (a 32 thread). Richiedendo alla CPU un numero di worker superiore a quelli fisicamente supportati (20), i tempi peggiorano per tutte le configurazioni a causa dell'overhead introdotto dal continuo *context switching*. In questo scenario sfavorevole, sebbene tutte le politiche subiscano un degrado,

la gerarchia rimane invariata, con `dynamic` che si conferma l'approccio più resiliente.

Impatto del Memory Layout: AoS vs SoA

L'ultimo scenario analizzato è l'impatto del layout dei dati in memoria. E' stato effettuato un confronto diretto tra l'approccio tradizionale orientato agli oggetti *Array of Structures* (AoS) e l'approccio orientato ai dati *Structure of Arrays* (SoA).

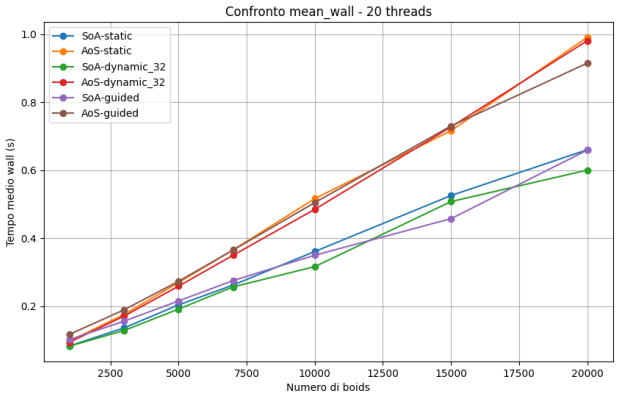

Figure 6: Confronto dei tempi di esecuzione tra layout AoS e SoA a 20 thread. Il divario prestazionale si allarga progressivamente all'aumentare della mole di dati (Boids), evidenziando il limite di banda dell'architettura AoS.

Come evidenziato nella Figura 6, per carichi di lavoro leggeri ($N = 1.000$) la differenza tra le due architetture risulta trascurabile. In questa fase, l'intera struttura dati è sufficientemente piccola da risiedere interamente nei livelli più alti e veloci della gerarchia di memoria. Tuttavia, all'aumentare della dimensione del dominio, il sistema esce dalla comfort zone della cache e il costo di accesso alla memoria RAM diventa il fattore dominante. A 20.000 boid, la versione AoS subisce un netto degrado. Al contrario, l'architettura SoA mitiga drasticamente questo collo di bottiglia.

Conclusioni

Il percorso di sviluppo ha mostrato come l'aggiunta di thread sia una condizione necessaria, ma non sufficiente, per ottenere alte prestazioni. La vera scalabilità si ottiene solo attraverso un utilizzo efficiente della gerarchia di memoria. L'adozione del layout *Structure of Arrays* (SoA) e del codice *branchless* ha

dimostrato che il modo in cui i dati vengono forniti alla CPU è altrettanto importante di quanti core vengano utilizzati. Grazie alle varie ottimizzazioni effettuate: partizionamento tramite griglia spaziale, architettura SoA e Lazy Spatial Sorting e vettorizzazione, il progetto ha centrato l'obiettivo di abbattere i colli di bottiglia del modello sequenziale. I vari benchmarking effettutati, hanno identificato una chiara **configurazione ottimale**: l'impiego di **20 thread** gestiti dallo scheduler `dynamic`. In questo assetto, il sistema ha fatto registrare uno *speedup* di picco pari a **7.98x** su un numero di 20.000 agenti.